TASK 02

DATA CLEANING & FEATURE ENGINEERING

Project Documentation Report

Dataset: Retail Store Sales Dataset

1. Project Overview

Data cleaning and feature engineering are important stages in a data science workflow. Raw datasets may contain missing values, inconsistent data types, unusual observations, and categorical information that cannot be used directly by many machine-learning algorithms. This task prepares a retail store sales dataset for analysis by inspecting its quality, treating missing values, investigating outliers, converting data types, encoding categorical variables, scaling numerical columns and creating additional useful features.

The work was implemented in a Jupyter Notebook using Python. The original dataset was kept unchanged, while separate DataFrame copies were created for cleaning, encoding, scaling, and feature engineering.

2. Objectives

- Inspect the structure and quality of a real-world retail transaction dataset.
- Identify and handle missing values and check for duplicate records.
- Detect potential numerical outliers and investigate whether they are valid.
- Convert date and identifier columns to appropriate data types.
- Encode categorical variables into numerical representations.
- Scale numerical features to a common range.
- Create at least five meaningful features from existing columns.
- Compare descriptive statistics before and after cleaning.
- Build and verify an automated preprocessing pipeline using Scikit-learn.

3. Tools and Technologies

Tool / Library	Purpose
Python	Main programming language used for data processing.
Pandas	Loading, inspecting, cleaning, and transforming tabular data.
NumPy	Numerical calculations and consistency checks.
Matplotlib	Visual inspection of numerical outliers using boxplots.
Scikit-learn	MinMaxScaler, imputers, OneHotEncoder, ColumnTransformer and Pipeline.

4. Dataset Description

The project uses the file retail_store_sales(in).csv. The dataset contains 12,575 transaction records and 11 columns. Each row represents a retail sales transaction.

Column	Description / role
Transaction ID	Unique transaction identifier.
Customer ID	Customer identifier.
Category	Product category.
Item	Name of the purchased item.
Price Per Unit	Price of one unit.
Quantity	Number of units purchased.
Total Spent	Total transaction amount.
Payment Method	Cash, credit card or digital wallet.
Location	Online or in-store transaction.
Transaction Date	Date of the transaction.
Discount Applied	Whether a discount was applied; missing values were retained as Unknown.

The transaction dates range from 1 January 2022 to 18 January 2025. The dataset has 12,575 rows and 11 columns before preprocessing.

5. Initial Data Inspection

The CSV file was loaded into a Pandas DataFrame. The notebook examined the dataset dimensions, column names, missing-value counts, duplicate rows, data types, sample records and descriptive statistics. This initial inspection established the baseline data quality before transformations.

Initial check	Result
Dataset shape	12,575 rows × 11 columns
Exact duplicate rows	0
Missing Item values	1,213 (9.65%)
Missing Price Per Unit values	609 (4.84%)
Missing Quantity values	604 (4.80%)
Missing Total Spent values	604 (4.80%)
Missing Discount Applied values	4,199 (33.39%)
Missing values in other columns	0

The missing-value percentages are calculated against all 12,575 records. No exact duplicate rows were found, so no records needed to be removed for duplication.

6. Handling Missing Values

Missing values were handled according to the type and meaning of each column. The original DataFrame was preserved and a copy named df_clean was used for cleaning.

Column	Treatment	Reason
Item	Filled with 'Unknown'.	Preserves the transaction while making the missing item name explicit.
Price Per Unit	Filled with the column median (23.00).	The median is less affected by extreme values than the mean.
Quantity	Filled with the column median (6.00).	Provides a representative value while retaining records.
Total Spent	First calculated as Price Per Unit × Quantity where missing; any remaining missing values filled with the median (108.50).	Uses the transaction relationship where possible, with a median fallback.
Discount Applied	Missing values replaced with 'Unknown'.	An absent discount value does not prove that a discount was or was not applied.

After these operations, all 12,575 rows were retained and the cleaned DataFrame contained zero missing values. The unknown discount status was deliberately preserved as a separate category instead of being incorrectly converted to True or False.

7. Duplicate Record Check

The notebook used the Pandas duplicated() method to count exact duplicate rows. The original dataset contained zero duplicate rows. The final cleaned dataset also contained zero duplicate rows. Therefore, no records were deleted during this step.

8. Outlier Detection and Investigation

Potential outliers were detected in Price Per Unit, Quantity and Total Spent using the Interquartile Range (IQR) method. For each numerical column, the first quartile (Q1), third quartile (Q3), and IQR were calculated. The lower and upper thresholds were calculated as Q1 − 1.5 × IQR and Q3 + 1.5 × IQR.

After the missing numerical values were imputed, the IQR check produced the following results:

Numerical column	Q1	Q3	IQR	Lower bound	Upper bound	Flagged values
Price Per Unit	14	32	18	-13	59	0
Quantity	3	8	5	-4.5	15.5	0
Total Spent	52	192	140	-158	402	60

The 60 flagged Total Spent values were all 410. The notebook investigated these records by comparing Total Spent with Price Per Unit × Quantity. Fifty-six of the 60 flagged records matched the calculation. Four records had missing original unit prices; after median imputation, their calculated totals could not be confirmed from the original price values.

The notebook did not delete or cap the 60 flagged values. This is an investigation-based decision: an IQR flag identifies an unusual value, not automatically an error. The maximum observed transaction amount of 410 is consistent with the maximum observed unit price of 41 multiplied by the maximum quantity of 10. The four records with missing original prices should be treated as unverifiable rather than automatically classified as incorrect. Retaining them avoids removing potentially valid high-value transactions.

Note: the task requirement asks to detect and handle outliers. The submitted notebook detects and investigates them, but does not apply a separate capping or removal operation. This is documented transparently so the implementation is not overstated.

9. Data Type Conversion

The Transaction Date column was converted from object/string to datetime using pandas.to_datetime with errors='coerce'. This enables date-based calculations such as extracting year, month and weekday. Transaction ID and Customer ID were converted to Pandas string data types because they are identifiers rather than quantities for mathematical calculations.

Column(s)	Data type after conversion
Transaction ID	string
Customer ID	string
Transaction Date	datetime64[ns]
Price Per Unit, Quantity, Total Spent	float64
Category, Item, Payment Method, Location, Discount Applied	object

The notebook found no invalid transaction dates. The date range remained 1 January 2022 through 18 January 2025.

10. Encoding Categorical Variables

Categorical variables were transformed so that machine-learning algorithms can work with them. The encoding was applied to a separate copy named df_encoded.

- Discount Applied: True was mapped to 1, False to 0 and Unknown to -1. The negative value preserves the distinction between an unknown status and the two known outcomes.
- Category, Payment Method and Location: one-hot encoding was applied using pandas.get_dummies(). Each distinct category becomes a separate binary column.

The encoded DataFrame contained 21 columns. The identifier columns and item name were retained as columns in this intermediate DataFrame; the automated pipeline later encoded the item name and other categorical features.

11. Feature Scaling

Min-Max scaling was applied to Price Per Unit, Quantity and Total Spent using Scikit-learn's MinMaxScaler. The transformation maps each selected feature to the range 0 to 1 using the observed minimum and maximum.

Scaling is useful when numerical features have different ranges, because it places them on a common scale. In the notebook, the scaled values were stored in a separate DataFrame named df_scaled. The minimum of each scaled column was 0.0 and the maximum was 1.0.

The original cleaned values were not overwritten by this scaling step; scaling was performed on the encoded copy.

12. Feature Engineering

Five new features were created from the cleaned transaction date, unit price and quantity. The feature-engineered DataFrame was named df_features.

New feature	How it was created	Why it is useful
Transaction Year	Extracted year from Transaction Date.	Supports yearly sales comparisons and trends.
Transaction Month	Extracted month number from Transaction Date.	Supports monthly patterns and seasonality analysis.
Day of Week	Extracted weekday name from Transaction Date.	Allows comparison of sales activity across weekdays.
Price Category	Price Per Unit grouped as Low (≤15), Medium (>15 and ≤30), or High (>30).	Makes unit-price levels easier to compare.
Quantity Category	Quantity grouped as Small (≤3), Medium (>3 and ≤7), or Large (>7).	Groups transactions by purchase quantity.

The feature-engineered DataFrame contained 16 columns: the original 11 cleaned columns plus the five new features. These features add useful time-based and grouped information without removing the original transaction fields.

13. Descriptive Statistics Before and After Cleaning

The notebook compared count, mean, standard deviation, minimum, quartiles and maximum for Price Per Unit, Quantity and Total Spent. The following table reports the key statistics reproduced from the supplied CSV and the cleaning operations in the notebook.

Feature	Stage	Count	Mean	Std. dev.	Min	Median	Max
Price Per Unit	Before	11,966	23.37	10.74	5	23	41
Price Per Unit	After	12,575	23.35	10.48	5	23	41
Quantity	Before	11,971	5.54	2.86	1	6	10
Quantity	After	12,575	5.56	2.79	1	6	10

Total Spent	Before	11,971	129.65	94.75	5	108.50	410
Total Spent	After	12,575	130.21	93.58	5	110	410

The count increased to 12,575 for all three numerical columns after missing-value treatment. The means and standard deviations changed slightly because missing values were imputed. The minimum and maximum values remained unchanged. The descriptive statistics provide a numerical check of how the cleaning process affected the data.

14. Automated Preprocessing Pipeline Using Scikit-learn

As a bonus task, an automated preprocessing pipeline was implemented. The pipeline started from the original dataset, converted Transaction Date to datetime, extracted Transaction Year, Transaction Month and Day of Week, and converted the discount values to consistent text values. Transaction ID, Customer ID and the original Transaction Date were removed from the model input because they are identifiers or because date information was represented by the extracted date features.

A numerical pipeline used SimpleImputer(strategy='median') followed by MinMaxScaler. A categorical pipeline used SimpleImputer(strategy='constant', fill_value='Unknown') followed by OneHotEncoder(handle_unknown='ignore'). ColumnTransformer applied the appropriate pipeline to numerical and categorical columns, and a Scikit-learn Pipeline combined the preprocessing steps.

Pipeline component	Function
SimpleImputer (numerical)	Fills missing numerical values using the median.
MinMaxScaler	Scales numerical features to a 0–1 range.
SimpleImputer (categorical)	Fills missing categorical values with 'Unknown'.
OneHotEncoder	Converts categorical values into binary indicator features.
ColumnTransformer	Applies numerical and categorical transformations to their respective columns.
Pipeline	Combines the preprocessing workflow into a reusable sequence.

The notebook reported 12,575 input rows and a processed output shape of 12,575 × 229. The verification step checked the transformed values and reported no missing values and no infinite values. The pipeline was fitted and applied to the dataset in the notebook; the notebook does not show a saved pipeline artifact or a train/test split.

15. Final Data Quality Results

Validation	Final result
Rows and columns in df_clean	12,575 × 11
Missing values	0 in every column
Exact duplicate rows	0

Transaction Date type	datetime64[ns]
Transaction ID and Customer ID types	string
Transaction date range	2022-01-01 to 2025-01-18
Feature-engineered DataFrame	12,575 × 16
Encoded DataFrame	12,575 × 21
Automated pipeline output	12,575 × 229
Pipeline missing / infinite value check	False / False

The final quality check confirms that the cleaned dataset has no missing values or exact duplicate rows. The date field has the expected datetime type, and the original record count has been preserved.

16. Conclusion

Task 02 successfully demonstrates the main stages of data cleaning and feature engineering on a retail store sales dataset containing 12,575 records. Missing values were treated using median imputation, calculated totals and an explicit Unknown category. Duplicate checks found no duplicate records. Potential outliers were identified with the IQR method and investigated rather than removed automatically. Data types were corrected, categorical variables were encoded, numerical columns were scaled and five meaningful features were created.

The bonus Scikit-learn pipeline combined numerical imputation and scaling with categorical imputation and one-hot encoding. Its output contained 229 processed features and passed the notebook's missing-value and infinite-value checks. The resulting data is more consistent and suitable for further exploratory analysis and machine-learning preparation.

A point for future improvement is to decide, with domain validation, whether the 60 high transaction totals should remain unchanged or be handled through a documented capping or correction rule. The current notebook flags and investigates these values but retains them.